



SOC Lab: Monitorización de Amenazas y Defensa Activa (IPS)

Autor: MAFO-sec

Proyecto: Ciclo completo de Ciberseguridad: Visibilidad (Superset) y Mitigación (Fail2Ban/CrowdSec).

1. Fase 1: Sistema de Monitorización (SOC)

1.1 Despliegue del Servidor Web "Sensor"

Instalamos Apache y creamos el formulario que generará los logs de ataque.

- **Archivo:** /var/www/html/index.php

```
PHP
<?php
$mensaje = "";
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $user = $_POST["user"] ?? "";
    $pass = $_POST["pass"] ?? "";
    if ($user === "admin" && $pass === "1234") {
        $mensaje = "Login correcto";
    } else {
        $mensaje = "Usuario o contraseña incorrectos";
    }
}
?>
```

Formulario de Login

Usuario:

Contraseña:

1.2 Preparación de la Base de Datos para Superset

Antes de mover los logs, debemos preparar el destino en Superset:

1. **Conectar la DB:** En el panel de Superset, ve a *Settings -> Database Connections*.
2. **Crear Tabla:** Asegúrate de tener creada la tabla `apache_logs` con los campos: `ip_address`, `datetime`, `method`, `url`, `status_code` y `size`.

The screenshot shows the 'Connect a database' dialog in Superset, specifically the 'STEP 2 OF 3' section titled 'Enter the required PostgreSQL credentials'. It includes fields for Host (superset_db), Port (5432), Database name (superset), Username (superset), Password (masked with dots), and Display Name (PostgreSQL). There are 'Back' and 'Connect' buttons at the bottom.

The screenshot shows the 'Database' configuration panel in Superset. It has a 'Database' dropdown set to 'postgresql' (PostgreSQL), a 'Schema' dropdown set to 'public', and a 'Table' dropdown set to 'apache_logs'. There are refresh icons next to the Schema and Table dropdowns.

The screenshot shows the Superset chart configuration interface. On the left, the 'Chart Source' is set to 'public.apache_logs'. Below it, there's a 'Search Metrics & Columns' bar. The 'Metrics' section shows 'COUNT(*)'. The 'Columns' section lists 'datetime', 'id', 'ip_address', 'method', and 'url'. On the right, the 'Data' tab is active, showing 'Dimensions' with 'ip_address', 'status_code', and 'datetime'. It also has 'Series limit' (None), 'Time series columns' (Time series columns), 'Row limit' (10000), and a 'URL' field. A 'Create chart' button is at the bottom.

1.3 Automatización con Python (El Pipeline)

Este script es el que "alimenta" a Superset leyendo el archivo `access.log` de Apache en tiempo real.

- **Dependencias:** `sudo apt install python3-psycopg2`
- **Archivo:** `procesar_logs.py`

```
import re
import psycopg2
from datetime import datetime

# Conexión a la DB de Superset
try:
    conn = psycopg2.connect(
        host="10.0.2.20", port="5432", database="superset", user="superset",
        password="superset"
    )
    cur = conn.cursor()
    log_pattern = re.compile(r'(\d+\.\d+\.\d+\.\d+) - - \[(.*?)\] "(.*?) (.*?) (.*?)" (\d+) (\d+|-)')

    with open('/var/log/apache2/access.log', 'r') as f:
        for line in f:
            match = log_pattern.match(line)
            if match:
                ip, dt_str, method, url, _, status, size = match.groups()
                dt = datetime.strptime(dt_str.split(' ')[0], '%d/%b/%Y:%H:%M:%S')

                size = 0 if size == '-' else int(size)
                cur.execute("INSERT INTO apache_logs (ip_address, datetime, method, url, status_code, size) VALUES (%s, %s, %s, %s, %s, %s)",
                    (ip, dt, method, url, int(status), size))

            conn.commit()
            print("¡Éxito! Logs insertados.")
except Exception as e:
    print(f"Error: {e}")
```

1.4 Visualización de Ataques en Superset

Configuramos los gráficos en Superset para ver los datos que inserta el script:

- **Gráfico de Barras:** IPs que generan más errores 401/403.
- **Gráfico Temporal:** Picos de tráfico generados por Hydra.

datetime	ip_address	status_code	COUNT(id)
2026-05-02	127.0.0.1	200	12
2026-05-02	10.0.2.1	200	6
2026-05-02	10.0.2.9	404	5
2026-05-02	10.0.2.9	200	5
2026-05-02	127.0.0.1	404	2
2026-05-02	192.168.1.132	200	2
2026-05-02	192.168.1.132	404	1



Fase: Defensa Activa con Fail2Ban e Infraestructura Proxy

En esta etapa, configuramos la respuesta ante incidentes. El objetivo es que, tras detectar los ataques visualizados en Superset, el sistema bloquee automáticamente la IP del atacante en el firewall del servidor.

1. Preparación de la Infraestructura

Para que el laboratorio sea realista y escalable, utilizamos una estructura de directorios organizada y un proxy inverso.

1.1 Estructura de Directorios

Ejecutamos el siguiente comando para preparar el entorno de trabajo:

```
mkdir -p
fail2ban-web/{app/templates,fail2ban/jail.d,fail2ban/filter.d,proxy,logs}
cd fail2ban-web
```

```
fail2ban-web/
├── docker-compose.yml
├── app/
│   ├── Dockerfile
│   ├── app.py
│   └── templates/
│       └── login.html
├── fail2ban/
│   ├── jail.d/
│   └── flask-login.local
```

1.2. El Backend de la Aplicación (app/app.py)

Crear el archivo que gestionará el login y generará los logs necesarios para la detección. Este script identifica la IP real del atacante incluso detrás del proxy.

```
from flask import Flask, request, render_template
from datetime import datetime
import os

app = Flask(__name__)

LOG_DIR = "logs"
if not os.path.exists(LOG_DIR):
    os.makedirs(LOG_DIR)

def get_client_ip():
    x_forwarded_for = request.headers.get("X-Forwarded-For", "")
    if x_forwarded_for:
        return x_forwarded_for.split(",")[0].strip()
    return request.remote_addr or "unknown"

def log_attempt(ip, status):
    now = datetime.now()
    filename = now.strftime("%Y-%m-%d") + ".log"
```

```

filepath = os.path.join(LOG_DIR, filename)
user_agent = request.headers.get("User-Agent", "-")
log_line = f'{ip} - - [{now.strftime("%d/%b/%Y:%H:%M:%S +0000")}] "POST /login
HTTP/1.1" {status} 0 "-" "{user_agent}"\n'
with open(filepath, "a", encoding="utf-8") as f:
    f.write(log_line)

@app.route("/", methods=["GET"])
def index():
    return render_template("login.html")

@app.route("/login", methods=["POST"])
def login():
    ip = get_client_ip()
    username = request.form.get("username")
    password = request.form.get("password")
    if username == "admin" and password == "admin123":
        log_attempt(ip, 200)
        return "Login correcto"
    else:
        log_attempt(ip, 401)
        return "Login incorrecto", 401

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

1.3. Interfaz de Usuario (app/templates/login.html)

Este es el formulario que verá el usuario (y el atacante). Es fundamental que el atributo `action="/login"` coincida con la ruta definida en nuestro `app.py`

```

<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Login</title>
</head>
<body>
    <h2>Login Test</h2>
    <form method="POST" action="/login">
        <input type="text" name="username" placeholder="Usuario"><br><br>
        <input type="password" name="password" placeholder="Password"><br><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>

```

1.4. Dockerización del Entorno

Para desplegar el laboratorio, necesitamos definir cómo se construirá la aplicación y cómo se comunicarán los contenedores.

Dockerfile (app/Dockerfile):

```

FROM python:3.12-slim

WORKDIR /app

RUN pip install --no-cache-dir flask

```

```
COPY app.py /app/app.py
COPY templates /app/templates
```

```
CMD ["python", "app.py"]
```

Docker Compose (fail2ban-web/docker-compose.yml)

Este archivo orquestrará el servidor Flask, el Proxy Nginx y el servicio de Fail2Ban, compartiendo el volumen de /logs.

```
services:
  app:
    build: ./app
    container_name: fail2ban-app
    restart: unless-stopped
    volumes:
      - ./logs:/app/logs
    expose:
      - "5000"

  proxy:
    image: nginx:alpine
    container_name: fail2ban-proxy
    restart: unless-stopped
    depends_on:
      - app
    ports:
      - "8080:80"
    volumes:
      - ./proxy/nginx.conf:/etc/nginx/nginx.conf:ro

  fail2ban:
    image: crazymax/fail2ban:latest
    container_name: fail2ban-engine
    restart: unless-stopped
    depends_on:
      - proxy
    network_mode: "service:proxy"
    cap_add:
      - NET_ADMIN
      - NET_RAW
    volumes:
      - ./logs:/logs:ro
      - ./fail2ban/jail.d:/data/jail.d:ro
      - ./fail2ban/filter.d:/data/filter.d:ro
    environment:
      - TZ=Europe/Madrid
      - F2B_LOG_LEVEL=INFO
      - F2B_DB_PURGE_AGE=1d
```

1.5. Configuración del Servidor Proxy (proxy/nginx.conf)

El archivo de configuración de Nginx permite que el tráfico llegue a la aplicación Flask y, lo más importante, gestiona las cabeceras de IP para que Fail2Ban no bane al propio proxy.

```
worker_processes auto;

events {
    worker_connections 1024;
}

http {
```

```

sendfile on;
resolver 127.0.0.11 ipv6=off;

# Confiar en IPs de Docker y OPNsense LAN para X-Forwarded-For
set_real_ip_from 172.18.0.0/16;
set_real_ip_from 10.0.2.0/24;
real_ip_header X-Forwarded-For;
real_ip_recursive on;

server {
    listen 80;
    server_name _;
    location / {
        proxy_pass http://app:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

2. Configuración de la Seguridad (Fail2Ban)

2.1. Definición del Filtro (fail2ban/filter.d/flask-login.conf)

Le indicamos a Fail2Ban qué patrón debe buscar en los logs generados por app.py.

```

[Definition]
failregex = ^<HOST> - - .* "POST /login HTTP/1\1" 401 -
ignoreregex =

```

2.2. Configuración de la Cárcel (fail2ban/jail.d/flask-login.local)

Establecemos las reglas del "castigo": 3 intentos fallidos en 60 segundos provocan un baneo de 10 minutos.

```

[flask-login]
enabled = true
filter = flask-login
logpath = /logs/*.log
maxretry = 3
findtime = 60
bantime = 600
port = 80,8081
banaction = iptables-multiport

```

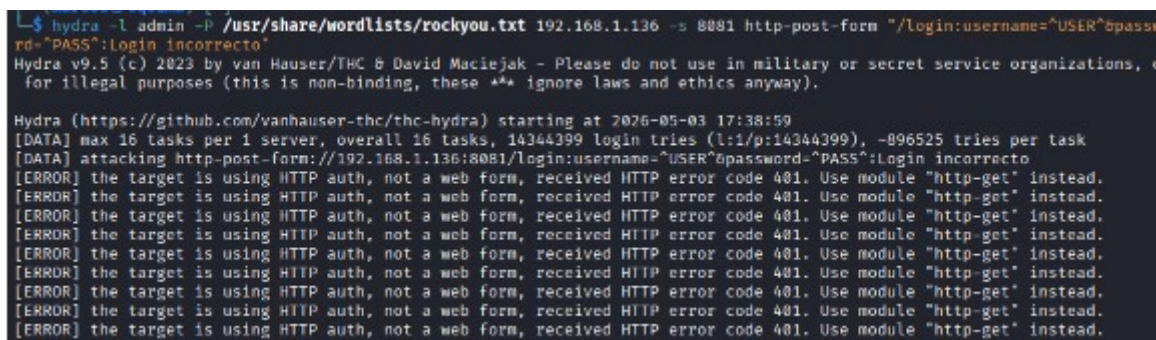
3. Verificación: El Ataque y el Baneo

Una vez levantado el entorno con docker-compose up -d, procedemos a la fase de pruebas.

3.1. Ejecución del Ataque (Hydra)

Desde nuestra máquina de ataque (Kali Linux), lanzamos un ataque de fuerza bruta dirigido al puerto 8081.

```
hydra -l admin -P pass.txt 192.168.1.136 http-post-form  
"/login:username=^USER^&password=^PASS^:incorrecto" -s 8081
```

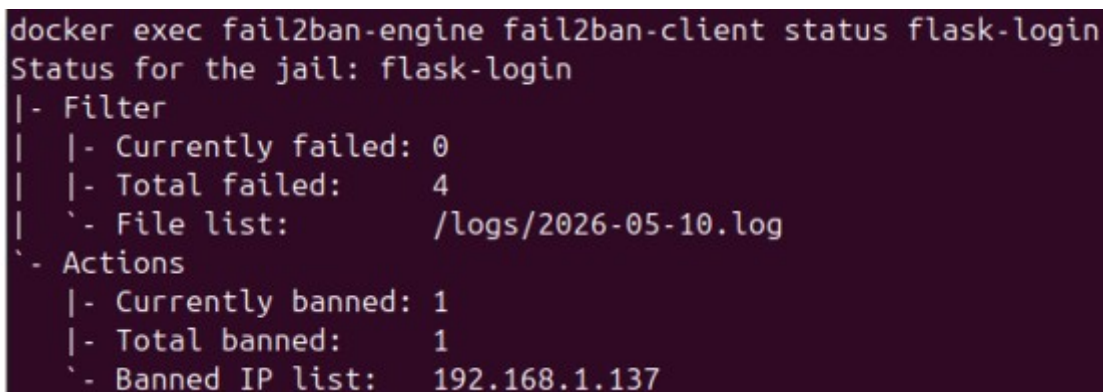


3.2. Confirmación del Baneo

Podemos verificar en tiempo real cómo Fail2Ban identifica la IP del atacante y la añade a la lista de bloqueados.

Comando de verificación:

```
docker exec -it fail2ban fail2ban-client status flask-login
```



Para desbanear

```
sudo docker exec fail2ban-engine fail2ban-client unban <IP_DE_TU_KALI>
```

```
sudo docker exec fail2ban-engine fail2ban-client unban --all
```


Fase: Defensa Moderna con CrowdSec (IPS Colaborativo)

En esta etapa, implementamos **CrowdSec**, una herramienta de seguridad que utiliza inteligencia colectiva. Si una IP es detectada atacando a otros usuarios de la comunidad, CrowdSec la bloqueará en nuestro servidor incluso antes de que intente atacarnos.

1. Conceptos Clave de CrowdSec

- **Agente:** Lee los logs y detecta comportamientos sospechosos (escenarios).
- **Bouncer (Gorila):** Es el encargado de ejecutar el baneo (en nuestro caso, a nivel de firewall/red).

2. Instalación y Configuración de CrowdSec

Para que CrowdSec funcione, dividimos el proceso en tres partes: el motor de detección, la adquisición de datos y el bouncer (cortafuegos).

2.1. Instalación del Motor de Detección

Primero, añadimos los repositorios oficiales e instalamos el agente y las colecciones de escenarios de ataque HTTP.

```
# Añadir repositorio e instalar
curl -s https://install.crowdsec.net | sudo sh
sudo apt install crowdsec -y
```

```
# Instalar colecciones de detección de ataques web
sudo cscli collections install crowdsecurity/http-cve
sudo cscli collections install crowdsecurity/base-http-scenarios
```

2.2. Configuración de la Adquisición de Datos (`acquis.yaml`)

Debemos indicar a CrowdSec dónde están los logs que genera nuestra aplicación Flask para que pueda analizarlos.

- **Archivo:** `sudo nano /etc/crowdsec/acquis.yaml`

```
filenames:
- /home/usuario/fail2ban-web/logs/*.log # <--- Ruta de los logs de la app
labels:
  type: nginx # Usamos el tipo nginx para que aproveche el formato CLF
---
filenames:
- /var/log/auth.log
labels:
  type: syslog
```

3. Instalación del Bouncer (El "Músculo")

El bouncer es el componente que escribe las reglas en el firewall (iptables) de Ubuntu para ejecutar el bloqueo efectivo.

3. Instalación del Bouncer (El "Músculo")

El bouncer es el componente que escribe las reglas en el firewall (iptables) de Ubuntu para ejecutar el bloqueo efectivo.

3.1. Instalación del Bouncer de Firewall

1. Generar la API Key:

```
sudo cscli bouncers add MiBouncerUbuntu
```

2. Configurar el Bouncer:

- **Archivo:** `sudo nano /etc/crowdsec/bouncers/crowdsec-firewall-bouncer.yaml`
- **Modificaciones:**
 - `api_url: http://127.0.0.1:8080/`
 - `api_key: <PEGA_AQUI_TU_LLAVE>`
 - En la sección `nftables:`, cambia `enabled: true` por `enabled: false` (para usar iptables).

3. Reiniciar Servicios:

```
sudo systemctl restart crowdsec  
sudo systemctl restart crowdsec-firewall-bouncer
```

4. Configuración del Entorno para Laboratorio

CrowdSec viene configurado por defecto para **ignorar** las IPs privadas (como las de tu red local 192.168.x.x) para evitar bloqueos accidentales. Para nuestro laboratorio, debemos desactivar esta protección.

4.1. Modificación de la Whitelist

Editamos el archivo de configuración de "listas blancas" para que CrowdSec nos permita banear a nuestra máquina Kali (IP local).

- **Archivo:** `/etc/crowdsec/parsers/s02-enrich/whitelists.yaml`

```
sudo nano /etc/crowdsec/parsers/s02-enrich/whitelists.yaml
```

```
whitelist:  
  reason: "private ipv4/ipv6 ip/ranges"
```

```
ip:
- "127.0.0.1"
cidr:
# - "192.168.0.0/16" # ← Comenta esta línea
# - "10.0.0.0/8"      # ← Comenta esta línea
# - "172.16.0.0/12"   # ← Comenta esta línea
```

```
sudo systemctl restart crowdsec
```

4.2. Adaptación del Backend para CrowdSec (app.py)

Para que CrowdSec pueda "entender" lo que pasa sin configuraciones adicionales, hemos modificado la función `log_attempt`. Ahora, los logs siguen el formato estándar de los servidores web profesionales.

¿Qué ha cambiado?

- **Formato de línea:** Se ha estructurado la cadena de texto para incluir la fecha entre corchetes [], el método HTTP, el código de estado (200 o 401) y el User-Agent.
- **Compatibilidad:** Este formato es reconocido automáticamente por el *parser* `s01-parse/apache2.yaml` de CrowdSec.

```
from flask import Flask, request, render_template
from datetime import datetime
import os

app = Flask(__name__)

LOG_DIR = "logs"
if not os.path.exists(LOG_DIR):
    os.makedirs(LOG_DIR)

def get_client_ip():
    x_forwarded_for = request.headers.get("X-Forwarded-For", "")
    if x_forwarded_for:
        return x_forwarded_for.split(",")[0].strip()
    return request.remote_addr or "unknown"

def log_attempt(ip, status):
    now = datetime.now()
    filename = now.strftime("%Y-%m-%d") + ".log"
    filepath = os.path.join(LOG_DIR, filename)
    user_agent = request.headers.get("User-Agent", "-")
    log_line = f'{ip} - - [{now.strftime("%d/%b/%Y:%H:%M:%S +0000")}] "POST /login\n'
    log_line += f'HTTP/1.1" {status} 0 "-" "{user_agent}"\n'
    with open(filepath, "a", encoding="utf-8") as f:
        f.write(log_line)

@app.route("/", methods=["GET"])
def index():
    return render_template("login.html")

@app.route("/login", methods=["POST"])
def login():
    ip = get_client_ip()
    username = request.form.get("username")
    password = request.form.get("password")
```

```

if username == "admin" and password == "admin123":
    log_attempt(ip, 200)
    return "Login correcto"
else:
    log_attempt(ip, 401)
    return "Login incorrecto", 401

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

5. Verificación de la Defensa Activa

5.1. Simulación de ataque

Lanzamos un bucle de ataques desde la máquina Kali:

```
for i in {1..10}; do curl -X POST http://192.168.1.136:8081/login -d "username=admin&password=wrong"; done
```

```

L$ # Desde Kali, 10 intentos rápidos
for i in {1..10}; do curl -X POST http://192.168.1.136:8081/login -d "username=admin&password=wrong"; done
Login incorrectoLogin incorrectoLogin incorrectoLogin incorrectoLogin incorrectoLogin incorrectoLogin incorrectoLogin incorrectoLogin incorrectoLogin incorrecto

```

5.2. Comprobación de Decisiones

Verificamos que CrowdSec ha tomado la decisión de bloquear la IP:

```
sudo cscli decisions list
```

ID	value	reason	country	as	decisions
	created_at	kind			
8	Ip:192.168.1.137	LePresidente/http-generic-401-bf			ban:
1	2026-05-10T16:30:36Z	crowdsec			
7	Ip:10.0.2.1	LePresidente/http-generic-401-bf			ban:
1	2026-05-10T16:27:29Z	crowdsec			
4	Ip:10.0.2.9	crowdsecurity/ssh-slow-bf			ban:

```
sudo cscli decisions
```

ID	Source	Scope:Value	Reason	Action
Country	AS	Events	expiration	Alert ID
30006	crowdsec	Ip:192.168.1.137	LePresidente/http-generic-401-bf	ban
		12	3h59m53s	8

5.3. Desbanear en CrowdSec

```

sudo cscli decisions delete --ip <IP_de_Atacante>
sudo cscli decisions delete --all

```

IMPORTANTE: Limpieza

Cuando se termines el laboratorio, no olvidar **devolver la whitelist** a su sitio para que tu servidor vuelva a ser "amigable" con tu red local:

Bash

```
sudo mv /tmp/whitelists.yaml /etc/crowdsec/parsers/s02-enrich/
```

```
sudo systemctl restart crowdsec
```